Replay Is a Family of Assertions: Fork-Safety in Event-Sourced Agent Runtimes

Yohei Nakajima
Untapped Capital / activegraph.ai

Revised research draft, August 2026

Abstract

Event-sourced agent runtimes commonly claim deterministic replay, cheap forking, and lineage. These are families of assertions, not one guarantee. A fixed-log fold may be deterministic while reactive settlement remains schedule-sensitive; a recorded oracle result can enable zero-call replay while the original interaction still changed cost or world state; and a prefix copy cannot undo an action in a discarded suffix.

We present an executable F# contract that separates facts from effect requests, makes scheduling explicit, and assesses a cut relative to projection, strict execution replay, external continuation, or a declared counterfactual-world observation. Preserved counterexamples show that general settlement is neither terminating nor confluent and that disjoint declared writes do not prevent read/trigger interference. Against 5,540 public runtime logs, all 317,296 projection cuts satisfy the stated observation, while 36,251 cuts split a model request and do not license continuation; another 120,969 are Conditional because a committed oracle result must be served from the record without re-execution. In 121 observed forks, 16,625 retained-prefix events have zero structural mismatches, but none crosses an external-effect boundary. A separate public offline conformance fork crosses a committed, recorded oracle boundary; its hash-bound receipt verifies that the inherited result was served with zero inherited external calls under an authenticated fixture environment. The result reframes cheap forking as a property-indexed semantic contract with explicit evidence and environment preconditions.

Artifact status. The code, checked-property suite, conformance vectors, and evidence ledger accompany this draft. The Synthetic Players capsule and the public activegraph-bridge post-oracle fixture are revision- and hash-pinned. The separate 24-run executive study has local source and release-bundle commits but no public remote. Its quantitative result is therefore retained only as an illustrative local finding and is excluded from the abstract until publication supplies external source provenance.

1 Introduction

Agent runtimes increasingly retain an append-only record of model calls, tool results, human decisions, state updates, and coordination events. Event sourcing makes several desirable properties seem immediate. If state is a fold over the log, a run can be replayed. If a child log shares a prefix with a parent, a run can be forked without recomputing that prefix. If causes are recorded, artifacts can be traced back to model and tool activity. ActiveGraph states these properties as consequences of making the log the source of truth [13].

This paper formalizes and empirically checks the replay and fork properties claimed on architectural grounds in Nakajima (2026). The relationship is deliberate: the earlier work describes

a deployed architecture, while the present work asks which side conditions make its architectural claims sound. Negative results about the originating runtime are therefore part of the contribution rather than exceptions to be hidden.

Two gaps motivate the work. First, "deterministic replay" often means deterministic replay *after oracle outputs have been fixed*. This is useful, but the phrase suppresses operational questions. Was the model output captured faithfully? Is a request retained without its result? Does replay serve the recorded result or make a new call? Is the runtime itself schedule-independent, or does a particular FIFO order merely produce one repeatable execution?

Second, event sourcing in a business application normally reconstructs application state; it does not claim to reverse the external world. That distinction becomes load-bearing for agents. A retrospectively discarded log suffix may contain an email, a charge, a delete, a publication, or a billable model call. The forked trace can omit the effect, but the world cannot be assumed to be one in which the effect never happened. A correct fork contract must say which property it preserves.

Figure 1 names seven assertions commonly compressed into the word "replay." They are ordered here from artifact identity toward claims about execution and the external world, but they do not form a simple implication chain. A system can preserve a decision while changing its path, or reproduce bytes while retaining too little in-log evidence to execute.

Assertion	Question answered
Byte	Is the serialized artifact identical under its verifier?
Trace	Are the same ordered semantic events retained?
Projection	Does folding those events yield equivalent selected state?
Path	Is the model, tool, and action sequence the same?
Decision	Is the task-level conclusion equivalent?
Execution	Can code re-execute with zero live calls by serving recorded results?
Environment	Does the observed external world match the world claimed by the prefix?

Figure 1: Replay is a family of assertions. Each row requires its own observation and evidence contract.

The familiar reducer shape

$$A : S \times E \to S \times F^* \tag{1}$$

is not the contribution. It is a Mealy-style transition system [11], resembles the Elm update architecture [4], and appears in functional event sourcing through the Decider pattern [2]. The operational seam needed here is a scheduled step over configurations plus an interpreter outcome tied back to a request identity:

$$\mathsf{step} : C \times \Sigma \to C, \qquad \mathsf{interpret} : R \to \{\mathsf{Committed}, \mathsf{Failed}, \mathsf{Unknown}\}. \tag{2}$$

The second function is outside the pure kernel. Evidence links its result to the request in the log. This makes scheduling, incomplete boundaries, and already-realized external effects explicit without claiming a new minimal agent calculus.

Table 1 states the relationship to the predecessor paper directly. The earlier architecture remains useful; the present paper supplies the contract under which its claims are licensed.

Contributions. This draft makes four contributions.

Claim in [13]	Status here	Qualification
Deterministic replay	Upheld, qualified	A fixed ordered log projects deterministically. Schedule-independent log production and zero-call execution require separate conditions (Sections 2–4).
Cheap forking	Upheld, qualified	Prefix copying is structurally cheap; continuation and external-world claims are property-relative (Sections 4 and 7).
End-to-end lineage	Upheld in the observed scope	The 121 observed children preserve compared prefix fields. Stronger replay licenses require request/outcome and provenance evidence (Sections 5 and 7).

Table 1: Claim status relative to *The Log is the Agent*.

1. It defines property-relative fork assessment for projection, strict execution replay, external continuation, and a declared external-world observation.

2. It separates effect evidence along external footprint, replay source, and lifecycle, allowing a recorded result and an irreversible footprint to coexist in one descriptor.

3. It gives an executable scheduler and preserves counterexamples showing that settlement need not terminate or converge. Disjoint declared writes remain insufficient when reads and emitted triggers interfere.

4. It checks the contracts against 5,540 public runtime logs, every cut in those logs, and 121 observed forks. A public offline conformance fixture additionally crosses a committed recorded oracle boundary and verifies zero inherited external calls. A separate 24-run local executive study is retained as illustrative evidence but excluded from the headline claim until its repository is public.

Genre. This is not a mechanized-proof paper. Model identities are stated as propositions; empirical claims are encoded as properties, checked over generated families with FsCheck, and evaluated exhaustively over the cuts of the available logs. We call these executable contracts or checked properties in the running text. A failed property produces a minimized, named regression fixture. Passing checks are finite evidence over the specified generators and artifacts, not universal proofs. This choice is intentional: calculi with machine-checked composition, termination, and information-flow results already exist [9, 6]. Our comparative advantage is an executable runtime contract connected to real runtime artifacts.

Roadmap. Section 2 defines the executable model and observations. Section 3 reports termination and confluence results. Section 4 defines fork properties and effect conditions. Section 5 connects those conditions to replayability grades. Sections 6 and 7 describe the artifact and empirical validation. We then position the work, discuss implications, and state limitations.

2 Executable Runtime Model

The model is intentionally small, but it is not a claim of expressive minimality. Its purpose is to make replay, scheduling, and effects observable enough to falsify claimed properties.

2.1 Events, projection, and identity

An event envelope is a tuple

$$e = (id, run, seq, kind, cause, emitter). \tag{3}$$

The event kind is a closed sum containing domain facts, signals, effect lifecycle events, and evidence facts. The evidence branch includes `HazardDetected(detail)`; an envelope records it as `EvidenceRecorded(HazardDetected detail)`, and the grader treats that fact as an explicit blocker. A log $L = \langle e_1, \ldots, e_n \rangle$ is ordered and append-only. The pure reducer $\mathsf{evolve} : S \times E \to S$ yields

$$\mathsf{project}(L) = \mathsf{fold}(\mathsf{evolve}, S_0, L). \tag{4}$$

Event identity is idempotent at the reducer boundary: if an event identifier has already been applied, a repeated envelope does not mutate state again. A duplicate identifier remains an integrity fault for fork assessment, because idempotent reduction must not silently certify a malformed trace.

State contains domain facts and signals, projected effect descriptors, evidence facts, integrity faults, and the set of applied event identifiers. When comparing projected domain state, the implementation excludes the bookkeeping set of applied identifiers. This prevents a replay that merely renames envelopes from being declared semantically different while still allowing exact-trace equality to detect the change.

Proposition 1 (Closed-log projection determinism)**.** *For a fixed ordered event log and a pure reducer, repeated evaluation of Equation 4 returns the same state. Oracle calls are not an exception: their recorded results, when present, are events in the fixed log.*

This proposition is the trivial base case. It says nothing about how the log was produced, whether the event order was unique, whether missing oracle results can be recovered, or whether effects may be executed again.

2.2 Behaviors and actions

A behavior is a tuple

$$b = (id_b, trigger_b, fire_b, W_b) \tag{5}$$

where $trigger_b : S \times E \to \mathbb{B}$ decides whether an event enables the behavior, $fire_b : S \times E \to A^*$ returns actions, and W_b is a declared write set used only as a diagnostic. Actions can emit a domain fact or signal, or request an external effect. Behaviors cannot emit an effect outcome or evidence attestation directly; those facts must cross the interpreter or validation boundary. This prevents a reducer from minting its own proof that an external action succeeded or that a run was verified.

The kernel does not execute actions. Emissions become pending events. Effect requests become pending request events and later block until an interpreter injects a committed, failed, or unknown outcome. Ordinary .NET tasks, network clients, mutable caches, and database adapters may exist outside the boundary; none are reachable from $trigger$ or $fire$.

2.3 Configurations and scheduling

A configuration contains projected state, pending events, enabled activations, outstanding effects, the retained trace, the behavior firing trace, and the next sequence and generated-identity counters.

A scheduler has two explicit selectors: one chooses a pending event, and the other chooses an enabled activation. Four policies are included: canonical, reverse-event, reverse-activation, and both reversed.

Writing a configuration as $\langle s, Q, A, O, L, T\rangle$, the two load-bearing transition rules are shown in Figure 2. The helper materialize turns a behavior's pure action descriptions into pending fact/signal events or effect-request events. It never produces an interpreter outcome.

$$\frac{e = \mathsf{selectEvent}_\sigma(Q) \quad s' = \mathsf{evolve}(s, e) \quad A' = A + \{(b, e) \mid \mathsf{trigger}_b(s', e)\}}{\langle s, Q, A, O, L, T\rangle \to_\sigma \langle s', Q \setminus e, A', O, L + e, T\rangle} \quad \textsc{Process-Event}$$

$$\frac{a = (b, e) = \mathsf{selectActivation}_\sigma(A) \quad X = \mathsf{fire}_b(s, e) \quad (Q', O') = \mathsf{materialize}(Q, O, X)}{\langle s, Q, A, O, L, T\rangle \to_\sigma \langle s, Q', A \setminus a, O', L, T + (b, e)\rangle} \quad \textsc{Fire-Activation}$$

Figure 2: Small-step scheduling rules. Event and activation selection are explicit. Emissions become visible to state only after a later Process-Event step, not within the activation batch for one trigger.

Processing a pending event first applies it to state and then computes enabled behaviors from the updated state. Enabled behaviors fire sequentially, but their emissions enter the pending queue. Activations handling the same triggering event therefore observe the same projected state. A behavior activated by a later-processed event observes changes committed by earlier Process-Event steps, including events materialized from prior activations. This cross-event ordering seam mirrors the one found in ActiveGraph and is sufficient for the read/trigger interference witness in Section 3.

Let $C \to_\sigma C'$ denote one legal event-processing or activation-firing step under scheduler σ. Settlement repeatedly steps until one of three outcomes is reached:

- **Quiescent**: no pending events, enabled activations, or outstanding effects;
- **BlockedAwaitingEffect**: no internal work remains, but at least one request has no terminal outcome;
- **Diverged**: a caller-supplied step bound is reached before either terminal outcome.

The step bound is checked only after testing for a terminal configuration. A run that becomes quiescent on the exact final step is therefore not misclassified as divergence. Conversely, reaching the bound does not prove a cycle; the kernel reports `StepBoundReached` and makes no stronger claim. BlockedAwaitingEffect is terminal for the settlement procedure but is not a quiescent normal form. The confluence definition in Section 3 ranges only over executions that reach Quiescent; uniqueness under blocking is a different property and is not claimed here.

2.4 Observations and equivalence

Confluence and fork claims are meaningless without specifying what is observed. The artifact implements three equivalences:

1. **ExactTrace**: envelope and event lists are structurally equal;
2. **ProjectedState**: domain projection is equal after excluding applied-event bookkeeping and envelope metadata;
3. **FactsOnly**(K): the selected fact keys agree.

The validation additionally reports byte equality when a source artifact provides it, decision equality when the study records decisions, and structural prefix equality modulo explicitly excluded lineage fields. These relations must not be substituted for one another. In particular, equal decisions do not imply equal paths, and equal projected state does not imply an unchanged external world.

3 Confluence and Quiescence

The ActiveGraph architecture describes behaviors reacting until quiescence without a privileged orchestrator. The working graph is then described as a deterministic projection of the log. Projection of a *fixed* log is deterministic, but the reactive loop can produce different logs under different legal schedules. We separate termination from uniqueness of the terminal projection.

Definition 1 (Termination at a bound)**.** *For behavior set B, initial configuration C_0, scheduler σ, and bound m, $\mathsf{settle}_m(B, C_0, \sigma)$ terminates if it reaches Quiescent or BlockedAwaitingEffect in at most m steps. StepBoundReached is evidence of bounded nontermination, not a proof of infinite reduction.*

Definition 2 (Schedule confluence under an observation)**.** *A behavior set is confluent for C_0 and observation P when every legal scheduler execution that reaches a quiescent configuration reaches a unique normal form modulo $\simeq_P$. Formally, if $C_0 \to^*_{\sigma_1} C_1$ and $C_0 \to^*_{\sigma_2} C_2$, and both C_1 and C_2 are quiescent, then $C_1 \simeq_P C_2$.*

This is Church–Rosser-shaped, but the current artifact does not prove a Church–Rosser theorem. It searches scheduler policies, generates restricted families, and preserves witnesses when the proposition fails. Newman's lemma connects termination plus local confluence to confluence in an abstract rewriting system [14]; our first counterexample removes the termination premise before the second examines uniqueness of quiescent forms.

3.1 A1: general termination is false

Counterexample 1 (Self-triggering emission)**.** *Let a behavior trigger on signal `loop` and emit the same signal. Seed the pending queue with `loop`. Every processed signal enables the behavior, and every firing enqueues another signal. No quiescent or effect-blocked configuration is reached. For every generated bound from 1 through 100, the artifact reports StepBoundReached at that bound.*

The example is intentionally small. It shows that purity of the reducer and append-only logging do not imply termination of a reactive runtime. Detecting a repeated finite fingerprint would not solve the general problem: generated identities and sequence counters can make configurations syntactically fresh while their domain behavior cycles. The implementation therefore avoids a heuristic `CycleDetected` verdict.

Two natural restoring contracts remain candidates. One assigns every internally emitted event a well-founded rank that strictly decreases along trigger edges. The other requires an acyclic graph from event kinds to behavior emissions. Both would exclude the fixture, but neither is claimed minimal, and both need refinement for state-dependent triggers.

3.2 A2: general confluence is false

Counterexample 2 (Overlapping writers)**.** *Behaviors α and β are both enabled by `start`. Each emits a write to fact `winner`; α writes 1 and β writes 2. Under canonical activation order the emitted events are processed so that the terminal winner is 2. Reversing activation order yields 1. Both executions quiesce, but their projected states differ.*

FsCheck generalizes this witness over generated integer values by constructing distinct adjacent values. Across the generated family, canonical and reverse activation schedules always disagree. The write-set diagnostic correctly reports `winner` as a conflict.

It is tempting to restore confluence by requiring declared write sets to be pairwise disjoint. That condition works for a deliberately restricted family: single-trigger behaviors that each unconditionally write a distinct key and do not read other behaviors' regions or emit cross-triggering signals. The suite generates up to eight such writers and checks all four scheduler policies; their projected states agree. The structural reason is stronger than those four samples: every generated key has exactly one unconditional writer, and no writer reads or triggers another, so any permutation applies the same set of commuting updates to distinct map entries. The four policies are executable regression coverage for that restricted family, not an enumeration of arbitrary runtime interleavings.

Declared disjointness is not sufficient in general.

Counterexample 3 (Read/trigger interference with disjoint writes)**.** *Two behaviors,* `left` *and* `right`*, trigger on* `start` *and emit signals* `left-ready` *and* `right-ready`*. Their declared write sets are empty. A third behavior triggers on* `left-ready`*, reads whether* `right-ready` *is already in state, and writes the only fact* `observed-right`*. All declared write sets are disjoint. Under canonical event order the observer records 0; under reverse event order it records 1.*

The second witness matters because a write-write conflict detector would certify it. The actual dependency crosses four surfaces: an emission, an event selection, trigger eligibility, and a state read. Any useful general restoring condition must constrain at least behavior reads, behavior writes, emitted event kinds, and the trigger relation. Under-declared dependencies must either be rejected or made observable by instrumentation.

Surface	Canonical event order	Reversed event order
Emission	`left-ready`, then `right-ready`	`right-ready`, then `left-ready`
Trigger	Observer becomes eligible before right is projected	Observer becomes eligible after right is projected
Read	`right-ready` absent	`right-ready` present
Write	`observed-right`=0	`observed-right`=1

Figure 3: Counterexample 3 crosses emission, event selection, trigger eligibility, and a state read despite disjoint declared write sets.

3.3 Production ordering masks the question

The ActiveGraph v1.10.0 scheduler uses a FIFO queue, evaluates matching behaviors in registration order, and dispatches them sequentially. Work due at the same delayed tick remains FIFO. This supplies a reproducible *ProductionOrder*, and it may be entirely appropriate as an operational contract. It is not evidence that alternate legal orders have the same normal form.

The distinction is practical. Projection occurs before scheduler notification, so an earlier registered behavior can emit an event whose projection changes the state read by a later behavior handling the same original event. Relation enumeration introduces another ordering seam when a backend query does not specify a canonical order. A stable FIFO queue can hide both effects in ordinary replay tests. Our scheduler tests bypass the production policy and vary event and activation order directly.

Result. The strongest supported conclusion is negative and conditional. General termination and general confluence are false. Isolated disjoint writers converge for the checked family, but declared writes alone do not restore confluence. Deriving and validating a general non-interference contract is future work; this draft does not present a minimal side condition.

4 Property-Relative Fork Safety

A trace fork at cut k retains $L_{<k}$ and gives subsequent events a child run identity. For a list representation,

$$\mathsf{fork}(L, k) = \mathsf{prefix}(L, k). \tag{6}$$

That definition makes prefix equality cheap, but it does not by itself make re-execution or external-world claims sound. We first describe effects, then define the properties a fork may be asked to preserve.

4.1 Effects along three dimensions

A flat partition into Pure, Idempotent, Cached, and OneShot combines questions that must be answered separately. A cached model response describes where replay material comes from; it does not describe whether the original call had external cost. We therefore use three orthogonal dimensions (Table 2).

Dimension	Values	Question
External footprint	Pure, Idempotent, Compensatable, OneShot, Unknown	What did execution do to the external world, and can that footprint be reconciled?
Replay source	Deterministic, Recorded, Uncaptured	Can replay reproduce or serve the result without consulting the live oracle?
Lifecycle	Requested, Committed, Failed, Unknown	Is the request/outcome boundary complete and trustworthy at this cut?

Table 2: Orthogonal effect dimensions.

Pure effects are internal descriptions with no external interaction. Idempotent effects can be repeated or overwritten under an explicit key. Compensatable effects require a separately verified compensation. OneShot effects are irreversible or conservatively treated as non-repeatable. Unknown fails closed for world-equivalence claims and becomes an explicit condition for record-served continuation. In the validation adapter, model calls are conservatively OneShot with respect to cost and oracle interaction. Their outputs may simultaneously be Recorded, allowing result replay without another call. These dimensions are orthogonal descriptions, not independent decision rules: a property may depend jointly on more than one dimension. In particular, Idempotent constrains repeated world mutation but does not guarantee that a second call returns the same bytes, timestamp, identifier, or model output.

Lifecycle is projected from events. A request must precede exactly one terminal outcome. Duplicate requests, conflicting descriptors, an outcome without a request, or a second outcome after terminal state are integrity faults. The first valid request and first valid terminal outcome remain the projected authority; malformed later events do not overwrite them.

4.2 Four fork properties

Definition 3 (ProjectionReplay)**.** *The child reconstructs the same domain projection as an independently folded copy of the retained, well-formed, ordered prefix. No execution occurs.*

Definition 4 (StrictExecutionReplay)**.** *Every retained effect result needed by execution is deterministic or recorded, and the cut does not retain a Requested or Unknown lifecycle. Replaying the prefix causes zero live oracle or tool calls.*

Definition 5 (ExternalContinuation)**.** *Relative to a caller-supplied premise that the target environment represents the retained prefix, the child may continue after strict replay. Inherited committed OneShot effects must be served from the record and never executed again. A committed Recorded effect with Unknown footprint is Conditional: it may be served without a live call, but no stronger world-state claim is licensed until its footprint is resolved. The v0 assessor takes the target-environment premise as an input assumption; it does not verify a snapshot or fresh-environment attestation. The companion bridge implementation supplies a concrete authenticated discharge path for the public conformance case in Section 7; the language-neutral assessor remains verifier-agnostic. This property deliberately does not assert that a retrospectively discarded source suffix left the original world unchanged.*

Definition 6 (CounterfactualWorld(Ω))**.** *Let Ω be a declared set of external observables, such as delivered messages, provider charges, published records, or payment-ledger entries. In addition to ExternalContinuation, the actual world and a world in which the discarded suffix did not occur agree under every observation in Ω. Discarded committed OneShot or unknown effects whose observables intersect Ω block the claim; discarded idempotent or compensatable effects whose observables intersect Ω make it conditional on reconciliation or successful compensation. Effects proven disjoint from Ω do not affect this property.*

The kernel does not inspect the external world. It checks whether retained effect evidence is sufficient to license CounterfactualWorld(Ω) under the stated footprint-coverage assumption. The current effect descriptor does not carry per-effect observable labels, so the executable v0 instantiates Ω as the entire external surface recognized by the selected adapter profile. Every classified non-Pure effect is therefore conservatively in scope. The companion bridge receipt can carry explicit observable tags, but the reported corpus counts do not retroactively use them. Supporting a narrower Ω in the generic assessor requires adapter-supplied tags; the reported counts are not claims about such a narrower set. Even a Sound verdict is a contract-relative license, not an empirical observation that two worlds are identical.

These definitions make "sound" property-indexed. The same cut may be Sound for ProjectionReplay, Conditional for ExternalContinuation, and Unsound for CounterfactualWorld. Table 3 summarizes the executable assessment.

4.3 Checked prefix identities

The executable suite checks the following trace identities over generated logs.

Proposition 2 (Identity fork)**.** $\mathsf{fork}(L, |L|)$ *retains the full trace. Under ExactTrace, the shared prefix is equal to L; child lineage is stored outside that shared list.*

Proposition 3 (Nested prefix collapse)**.** *For $j \leq i \leq |L|$, retaining i events and then j events has the same shared prefix as retaining j events directly. The nested branch still records the outer child as its parent, so raw branch records are not identical.*

Property	Retained-prefix conditions	Discarded-suffix conditions
ProjectionReplay	Whole source log is structurally well formed; ordering authority is fixed	None for projected prefix equality
StrictExecutionReplay	No open/unknown lifecycle; every needed result is deterministic or recorded	None
ExternalContinuation	Strict replay; retained committed OneShot effects are not re-executed; committed Recorded Unknown footprint is Conditional	Target environment represents the retained prefix as a caller-supplied premise
CounterfactualWorld (Ω)	ExternalContinuation conditions; effect classification covers Ω	No unreconciled footprint intersecting Ω; in-scope OneShot or unknown blocks

Table 3: Property-relative fork conditions.

Proposition 4 (Projection commutation)**.** *For a well-formed ordered log,*

$$\mathsf{project}(\mathsf{fork}(L, k)) = \mathsf{project}(\mathsf{prefix}(L, k)). \tag{7}$$

This is a trace/projection statement, not an external-world theorem.

Proposition 5 (Suffix irrelevance is property-relative)**.** *The discarded suffix is irrelevant to ProjectionReplay of the retained prefix. It is not generally irrelevant to CounterfactualWorld.*

The last proposition corrects an over-broad formulation of cheap forking. A committed OneShot event in the discarded suffix is a concrete witness: the forked log excludes it and projects correctly, but the action remains true in the world. Likewise, a discarded request without a trustworthy terminal outcome is Unsound because the external system may have executed it even though the log never learned the result.

4.4 Cuts through effects

For each log, the validator constructs every prefix state in one forward scan and every environmental suffix finding in one reverse scan. It then assesses all $|L|+1$ cuts. Three boundary cases are especially important.

Cut	Retained prefix	Licensed claim
k_0 before request	No inherited request	Projection is sound; a later source-side effect may still block a retrospective world claim
k_1 after request, before outcome	Requested lifecycle only	Strict replay and continuation are refused
k_2 after recorded commit	Request plus terminal recorded result	Strict replay can be sound; continuation requires zero re-execution

Figure 4: A single request/outcome pair creates three semantically different fork regions.

Before a request. The retained prefix does not inherit the effect. Projection replay is sound. If the source later committed a OneShot effect, CounterfactualWorld is unsound for a retrospective fork because that discarded action already occurred.

Between request and outcome. The prefix contains a Requested lifecycle. StrictExecutionReplay and ExternalContinuation are unsound: the effect may be running, may have committed without acknowledgement, or may have failed. A step bound or missing response must not be treated as a safe cancellation.

After a committed OneShot. The response may be Recorded, making strict replay possible with zero external calls. ExternalContinuation is Conditional: the interpreter must serve the recorded result and enforce zero re-executions. This is a semantic obligation, not merely an optimization.

After an uncaptured idempotent result. Idempotence alone does not reconstruct the retained result. Re-executing the same key may leave the external resource unchanged while returning a different receipt, timestamp, identifier, or oracle output. Because ExternalContinuation in this paper includes StrictExecutionReplay, the v0 verdict remains Unsound. A separate live-resumption property could permit a repeated call under a provider-specific outcome-equivalence contract; it is not the property checked here.

After a reported failure. A terminal Failed event does not establish that the external world was unchanged. A non-atomic remote operation may partially mutate state before reporting failure. The executable assessor therefore makes discarded failed Idempotent or Compensatable effects Conditional on reconciliation, and treats discarded failed OneShot or Unknown effects as Unsound. Only a failed Pure effect adds no world finding.

Compensation is likewise not retroactive. Even successful compensation may leave a temporal interval in which the mutation was observable and could have triggered webhooks, quotas, or other secondary effects. A Conditional Compensatable verdict licenses CounterfactualWorld only for an Ω and environment contract that explicitly exclude or reconcile those exposures.

These cases expose why a Decider that returns only domain events cannot express the full agent-runtime question. The log needs request/outcome boundaries, footprint evidence, and replay-source evidence before it can license execution or world-level claims.

5 Replayability Grades as Evidence Licenses

The activegraph-bridge project introduced five replayability levels from observed through native. The names arose from implementation failure modes. We make their meaning explicit by assigning each grade a set of licensed questions (Table 4).

Let $Q(g)$ be the set of questions licensed by grade g. The current taxonomy is defined as the chain

$$Q(Observed) \subset Q(Envelope) \subset Q(Boundary) \subset Q(Checkpointed) \subset Q(Native). \tag{8}$$

This chain is a definitional summary, not a substantive theorem. The artifact checks its ordering mechanics, but the paper's claim is narrower: a grade summarizes the questions supported by retained evidence, while a cut-level assessment decides one proposed fork. A richer taxonomy could contain incomparable evidence sets.

Grade	Required evidence	Licensed use
Observed	Some inspectable trace or result	Inspection and available lineage only
Envelope	Faithful invocation envelope and completion, no lossy-envelope fact	Recorded-output playback; fork before invocation
Boundary	Envelope; mediated effects; clean reconstruction; complete known effect boundaries; no blockers	Re-execution using recorded effects; forks at effect boundaries
Checkpointed	Boundary plus recorded checkpoint	Resume from a restorable snapshot
Native	Checkpointed plus native-runtime evidence	Native replay, fork, and structural diff

Table 4: Replay grades and their licenses.

5.1 Computation from the log

The grade is computed from projected evidence and effect completeness. It is not accepted as a source label. In particular, a `NativeRuntime` marker alone does not grant Native: the log must also satisfy checkpoint, boundary, and envelope prerequisites. Unknown footprint, uncaptured results, incomplete or unknown lifecycles, integrity faults, lossy envelopes, unmediated effects, and explicit hazards are blockers.

The adapter follows the same fail-closed rule. It grants evidence only for exact recognized source event types. An event whose name merely contains `verification` remains an unclassified domain signal. A verification event must contain an explicit Boolean verdict. The bridge-specific Boundary derivation additionally requires an effects-served count and an explicit null divergence: the source payload's `divergence` field must be present and its JSON value must be `null`, asserting that the bridge verifier detected no unresolved reconstruction divergence. This is logged evidence, not an independent proof of equivalence. These requirements prevent an event-name coincidence from raising the grade.

5.2 Grades are not monotone in time

It is natural to view grades as achievements that only rise. That view is incorrect for evidence predicates.

Counterexample 4 (Hazard downgrade)**.** *A prefix records a faithful envelope, completion, mediated boundary, clean reconstruction, and successful verification. It grades Boundary. Appending a* `EvidenceRecorded(HazardDetected detail)` *event adds an explicit blocker, so the extended log grades Envelope.*

The grade of a *fixed log or prefix* licenses questions about that evidence. It need not be monotone under extension. This matters for forks: the grade of the shared prefix can be higher or lower than the grade of the completed source run, and a later hazard must not retroactively be ignored.

5.3 Connection to fork safety

The grade is a compact mechanism for checking some fork preconditions, but it does not replace property-relative assessment. Boundary evidence suggests that effect results and reconstruction are available, licensing forks at known effect boundaries. A specific cut can still split a request

from its outcome. Likewise, a Boundary run can contain a committed OneShot effect that makes a CounterfactualWorld claim conditional or unsound.

The validation therefore reports both the run-level grade and all cut-level verdicts. A grade answers "what kinds of questions may this evidence support in principle?" A fork assessment answers "does this particular cut satisfy the conditions for property P?"

6 Executable Specification and Method

The artifact is a single .NET solution written in F#. Purity is enforced at the transition boundary, not imposed on the surrounding program. The source contains ordinary file I/O, JSON parsing, hashing, mutable library internals, and command-line orchestration outside the reducer. There is no database, dependency-injection container, actor framework, plugin system, network client, or asynchronous interpreter inside the kernel.

6.1 Artifact structure

The artifact includes:

- a closed event and effect model;
- a pure projector and explicit settlement scheduler;
- branch construction with parent/child identity, cut, shared prefix, continuation, inherited effects, and prefix hash;
- cut-level fork assessments and replay-grade computation;
- an ActiveGraph/bridge JSONL normalizer and batch validator;
- 50 passing tests, including FsCheck properties and named fixtures;
- seven language-neutral JSON conformance vectors and a JSON Schema;
- read-only SQLite export and actual-fork audit scripts;
- a findings log that records failed checked properties as they are discovered.

All F# projects treat warnings as errors. Pattern matches over closed domain unions are exhaustive. External effect execution is not linked into the kernel, so replay cannot accidentally call a model or tool through an injected client.

6.2 Property-based checks

FsCheck generates bounded event values, writer families, effect dimensions, and step bounds. Scheduler policies are enumerated rather than sampled because the executable model exposes four intended extremes. Properties include:

- deterministic projection and event-identity idempotence;
- exact-bound quiescence and bounded self-cycle divergence;
- convergence of isolated disjoint writers under all schedulers;

- schedule dependence for generated conflicting writers;
- identity, nested-collapse, projection, and suffix identities for forks;
- verdict matrices across generated footprint and replay-source values;
- grade-chain mechanics and prerequisite enforcement.

FsCheck shrinking is useful for scalar values, but the semantically important counterexamples are named and checked in as JSON-backed fixtures. The fixtures include overlapping writers, read/trigger interference despite disjoint writes, a self-cycle, a discarded OneShot effect, an unresolved request, malformed effect transitions, and forged look-alike evidence types.

6.3 Real-log normalization

Validation accepts ordered JSONL. For SQLite-backed ActiveGraph stores, an export script uses `events.seq` as the ordering authority and emits one file per run. Source domain events that are not part of the closed effect or evidence vocabulary remain signals and are reported by type. The adapter does not infer effects from every `*.requested` event because real domains use names such as `round.requested`; only a closed list of known external request types is classified.

This choice creates an explicit adapter-coverage premise. Unknown footprint fails closed once an event is classified as an effect, but an unclassified source event is not automatically treated as an effect. A Sound continuation verdict therefore means "Sound under the profile declaration that the listed unclassified types are domain-only." A new source type that hides an external interaction can invalidate that verdict. The validator reports every such type so the declaration can be audited rather than silently assumed.

For an external request, footprint is read from explicit side-effect metadata. A write without an idempotency key is OneShot; an unknown tool footprint remains Unknown. Model and embedding requests default conservatively to OneShot. A response is Recorded only when the log contains an explicit response hash, output hash, or raw output from which the validator derives a content SHA-256. The derived hash certifies captured replay material; it is not a provider receipt or proof of external commitment.

Every source event is assigned a contiguous normalized sequence. Bridge events can produce multiple evidence facts; this is why the bridge normalized count exceeds its source count. The source event identity is retained for the first fact, and deterministic derived identifiers are used for additional facts. Duplicate or non-contiguous normalized identity remains a blocker. The normalizer also retains the cumulative cut after each atomic source event. These source-boundary positions are reported separately from diagnostic cuts between co-derived facts.

6.4 All-cut enumeration

For a normalized log of length n, the validator evaluates cuts $0, \ldots, n$. Prefix states are computed with one scan; environmental findings for suffixes are accumulated in reverse. It reports both (i) every normalized position, useful for checking the executable transformation, and (ii) only source-event boundaries, which are the physically meaningful fork candidates in the source trace. The validator reports Sound, Conditional, and Unsound totals for ProjectionReplay, ExternalContinuation, and CounterfactualWorld, plus the number of runs containing an unsound counterfactual cut.

Actual ActiveGraph forks require a second audit because the child trace has lineage and store-level sequence differences. The SQL audit resolves the parent cut event, orders the parent prefix

and child trace independently, and compares corresponding event ID, type, actor, payload, frame, cause, and timestamp. In this SQLite schema, `events.seq` is the database-wide autoincrement primary key, not a run-local envelope field. It is used to order each trace and excluded from equality because copied child events receive new store positions. The normalizer then assigns the contiguous per-run sequence used by Equation 3; that derived ordinal is not a second physical field in the fork audit. Child run ID is also excluded. The audit separately counts retained external requests and unresolved boundaries.

6.5 Reproducibility discipline

Evidence entries record repository, immutable revision when available, source path, artifact hashes, counts, parser profile, command, publication status, and limitations. Checked-in fixtures are hash-bound and explicitly marked synthetic or sanitized. Private and local runtime payloads are not copied into the repository merely to make a test convenient. Missing revision or source provenance is reported as a limitation rather than replaced with a hash-only claim of reproducibility.

The local executive study follows a separate repository rule: generated developer artifacts remain ignored, while a deliberate redacted release bundle carries the store, metrics, source hashes, and verifier. The source and bundle now have local Git revisions. They are not treated as public evidence until a remote publishes those revisions.

7 Validation Against Runtime Artifacts

We evaluate the contracts against one public trace corpus and one public post-oracle conformance fork, and retain one local instrumented study as illustrative evidence. The public, sealed Synthetic Players capsule contains an ActiveGraph SQLite store. The public activegraph-bridge fixture executes an actual bridge fork after a committed recorded fixture-oracle effect. The local executive study records explicit boundary verification but has no public source remote. All three contain synthetic experimental content; "real" here means actual runtime artifacts rather than invented shape fixtures. None is independent of the ActiveGraph family, and we do not claim human behavioral validity or real-model performance.

7.1 Synthetic Players capsule

The public source is pinned to revision `82556ef9dae3`. The compressed engine store begins SHA-256 `7b7fdfe64b5b`, and the capsule checksum ledger begins SHA-256 `6d80f3b19b92`. The evidence ledger records the complete revision and digests. Running the public verifier reports:

> CAPSULE VERIFICATION PASS — 4,919 archived Phase 3–5 runs verified (4,916 confirmatory + 3 legacy diagnostics).

The decompressed database contains 5,540 runs and 311,756 events. Event types include 36,251 `llm.requested`, 36,227 `llm.responded`, and 30,397 `decision.parsed` records. The difference between request and response counts is reflected rather than repaired by the adapter.

The two run counts have different denominators. The store contains 5,505 completed and 35 incomplete runs. The public verifier covers 4,916 confirmatory Phase 3–5 runs plus three completed legacy diagnostics. The remaining 586 completed runs are outside that replay contract; the 35 incomplete runs are also outside it. Thus $4{,}919 + 586 + 35 = 5{,}540$.

7.1.1 Replay grades

All 5,540 logs grade Observed, and zero are verified *from the run log alone*. This does not contradict the passing capsule verifier. The capsule attestation is external to each run's event stream. A grading function that claims to depend on the log alone cannot silently import that attestation. This is a useful negative result: sealed-artifact reproducibility and self-describing run replayability are distinct properties.

7.1.2 All-cut results

Table 5 reports every cut. The projection total is $311{,}756 + 5{,}540 = 317{,}296$, because each run of length n has $n + 1$ cuts.

Property	Sound	Conditional	Unsound
ProjectionReplay	317,296	0	0
ExternalContinuation	160,076	120,969	36,251
CounterfactualWorld	150,230	24,172	142,894

Table 5: All-cut verdicts for 5,540 Synthetic Players logs.

Every projection cut is sound under the specified domain-projection observation. The 36,251 ExternalContinuation-unsound cuts equal the number of open/cut-through LLM requests. Direct sequence audit explains the equality: 36,227 requests are immediately followed by their linked response, so each creates exactly one cut between request and response; the remaining 24 requests are the final event of failed runs, so each also creates exactly one open cut. Conditional continuation cuts retain a committed OneShot oracle interaction and require serving the recorded result without another call. CounterfactualWorld is stricter: 4,923 runs have at least one Unsound cut because a discarded oracle interaction already occurred or a boundary is ambiguous.

The 160,076 Sound ExternalContinuation cuts validate only the no-inherited-effect case. A direct per-run ordinal query confirms that every one lies before the first classified external request, including all cuts in the 617 runs with no classified request. The Synthetic Players corpus alone therefore does not empirically validate a post-oracle continuation, target-environment attestation, or zero-reexecution receipt. Generated properties and named fixtures exercise post-oracle verdicts and typed no-reexecution obligations; the public conformance fork below separately exercises their runtime discharge under a fixture trust root.

The validator leaves domain and infrastructure types unclassified, including `behavior.started` and `decision.parsed`. It likewise leaves `infra.*`, `round.requested`, and `run.completed` unclassified. In particular, `round.requested` is not misclassified as an external request merely because its name ends in `requested`. Continuation verdicts are therefore relative to the audited profile declaration that these reported types are domain or infrastructure signals rather than hidden external effects.

7.1.3 Observed forks

The SQLite run table contains 121 child runs from 41 distinct parents and no nested child in this sample. All forks cut at a `round.played` event. The audit compares 16,625 parent-prefix events with the beginning of their child traces and finds zero mismatches in event ID, type, actor, payload, frame, cause, or timestamp. Every cut event resolves. Run identity and `events.seq` are excluded because they are expected to differ: in this store schema, `seq` is the database-wide autoincrement

key used for ordering, not a run-local field. The per-run normalized sequence is derived after export and was therefore not an additional physical field available for comparison.

None of the 121 retained prefixes contains a classified external request, and none contains an unresolved request. The observed forks therefore satisfy the current ExternalContinuation precondition for a domain-only cut family. This is strong evidence for cheap structural branching where the runtime actually forked. It is not evidence for an effect-boundary fork: the observed sample does not contain one.

7.2 Public post-oracle conformance fork

The companion activegraph-bridge artifact is public at revision `8855d3a9e779`. The complete public post-oracle fixture and verifier are revision-pinned. It records an actual bridge child fork after a committed effect classified as OneShot footprint plus Recorded replay source. The source run makes one deterministic offline fixture-oracle call. Verification and the child serve the committed outcome from the record, and the child diverges only after a changed tool result.

The receipt binds the parent and child run identities, cut boundary, parent log hash, exact retained-prefix hash, child log hash before receipt emission, inherited request and outcome identities, response hash, source and target runtime fingerprints, and target-environment claims. Its verifier checks those hashes, the zero-call counter, and an HMAC-SHA256 signature under a configured trust root whose claims include the child, cut, prefix, and target fingerprint. The published command reports:

> POST-ORACLE FORK RECEIPT PASS — committed oracle served from record; 0 inherited external calls.

The parent JSONL digest begins `884e8ae34296`; the child begins `747305a6774a`. The receipt file begins `da5503a80b92`, and its canonical internal receipt hash begins `5d36df7dc177`. The verified receipt names inherited request `evt_008`, its recorded outcome `evt_009`, one source oracle call, and zero inherited external calls in the fork.

This discharges the prior implementation gap for a public conformance case. An inherited recorded oracle outcome can cross a fork boundary without a second call, and the target environment premise can be authenticated against a configured trust root. The oracle is deterministic and offline, and the HMAC key is deliberately published as a fixture key. The result does not attest a real provider, production identity, model quality, or production environmental fidelity.

7.3 Illustrative local executive study

The second artifact contains 12 synthetic vendor due-diligence cases evaluated under two deterministic offline mock strategies, for 24 runs and 2,592 source events. The manifest states that no real LLM or external provider was used and limits the claim to instrumentation feasibility. It records fresh-factory reconstruction and enables boundary verification.

The artifact files are hash-bound. Hash prefixes for the run store, manifest, metrics, and paired results are `7a38a1e8`, `da1cbc3b`, `9f9242e1`, and `f4d95dbb`, respectively; the evidence ledger records the complete hashes. Review exposed that the containing migration-lab had no commit or remote. We repaired the local half of that defect: source revision `54dfde4d7379` and release-bundle revision `201efa7` bind a redacted manifest, metrics, pairs, report, compressed run store, and verifier. The repository still has no configured remote. This result remains illustrative, is excluded from the abstract, and is not externally source-reproducible until those revisions are published.

7.3.1 Decision and path equivalence

The paired artifact reports 66.6667% final-decision agreement and 0% ordered tool-path agreement. Eight of twelve pairs reach the same decision via different paths, yielding 66.6667% same-decision/different-path. This is not a model-quality result—both strategies are deterministic mocks. It is an instrumented witness that decision equivalence and trace equivalence answer different questions. A system that reports only agreement can hide total path divergence; a system that requires exact paths can reject decisions that are equal under a task-level observation.

7.3.2 Grades and cuts

Each run's source events normalize to two additional evidence facts: fresh reconstruction and mediated boundary. The 2,592 source events therefore become 2,640 normalized events. All 24 logs grade Boundary and all 24 are verified from explicit log facts. The 48 additional facts create 48 diagnostic intra-source positions that cannot be selected as cuts in the atomic source log. We therefore report source-event-boundary verdicts as the primary bridge result and retain every-normalized-position counts as a transformation check.

Property	Sound	Conditional	Unsound
ProjectionReplay	2,616	0	0
ExternalContinuation	1,968	264	384
CounterfactualWorld	0	264	2,352

Table 6: Source-event-boundary verdicts for 24 bridge logs.

All 2,616 source-boundary ProjectionReplay cuts are Sound; the diagnostic over every normalized position is 2,664 Sound. For completeness, the normalized ExternalContinuation totals are 1,992 Sound, 288 Conditional, and 384 Unsound; the CounterfactualWorld totals are 0 Sound, 288 Conditional, and 2,376 Unsound. Every run contains a OneShot `submit_recommendation` effect. No cut is unconditionally Sound for CounterfactualWorld: before the commit, the effect lies in the discarded suffix; after the commit, continuation is Conditional on not executing it again. The result demonstrates why Boundary grade licenses effect-boundary operations but cannot promise a world in which discarded actions never happened. These counts use the adapter-wide $\Omega_{profile}$, under which `submit_recommendation` is in scope. The public bridge v0.2 receipt records per-effect observable tags, but the legacy executive-study adapter does not expose such tags to this analysis. A narrower observable set could change the world verdicts; we do not report that unmeasured result.

7.4 Validation status

V1: compute grade distributions. Completed. The public capsule yields Observed for 5,540 logs because verification is external to the log; the bridge yields Boundary for 24 logs because the necessary evidence is explicit. The contrast supports log-derived grading rather than profile-derived labels.

V2: inspect real forks and their preconditions. Completed for the public capsule and the public conformance fixture. All 121 observed corpus forks preserve the compared prefix fields and retain no classified external request. The companion fixture tests one actual bridge fork after a committed recorded oracle call and verifies zero inherited external calls plus an authenticated

fixture-environment premise. It is conformance evidence, not a real-provider or production-attestor result.

V3: test production confluence conditions. Partially completed. Source audit confirms that ActiveGraph imposes FIFO plus registration order, which can mask non-confluence. The property suite bypasses that order. A complete inventory of production behavior reads, writes, emissions, triggers, and relation-order dependencies was not available, so we cannot certify or falsify a general production non-interference contract.

Combined interpretation. The evidence supports deterministic projection, structurally correct observed prefix copying, and one public post-oracle continuation under a verified conformance environment. It does not support an unrestricted claim that any historical cut is safely executable or environmentally counterfactual. Those stronger claims require effect-complete evidence and a property-specific precondition.

8 Related Work and Novelty Boundary

Reducers and event sourcing. Mealy machines produce output and a next state from a current state and input [11]. Elm's update loop maps a message and model to a new model and command [4]. Chassaing's functional event-sourcing Decider separates decision and evolution and demonstrates composition in F# [2]. These precedents invalidate a novelty claim for the reducer shape.

Fowler's event-sourcing account makes an ordered event sequence the basis for rebuilding application state and temporal queries, while explicitly warning that replayed updates to external systems can go wrong unless gateways suppress them [5]. Our work does not claim to discover that external effects complicate replay. It supplies a property-indexed cut assessment using request, outcome, footprint, and replay-source evidence in archived agent traces.

Reactive rules, termination, and confluence. The closest classical precedent for our scheduler counterexamples is the active database literature. Aiken, Hellerstein, and Widom analyze whether production rule sets terminate, produce a unique final database state, and produce a unique observable action stream [1]. Paton and Díaz survey ECA rule languages and the execution-model choices that determine active-system behavior [15]. Consequently, "rules can trigger rules and be order-sensitive" is not our novelty claim.

Newman's lemma relates termination and local confluence in abstract rewriting systems [14]. CRDTs provide confluence by construction for data types whose concurrent updates satisfy the required convergence conditions [18]. We neither prove a minimal general condition nor compete with these results. Our narrower contribution is to expose the ordering dependency in an agent runtime, preserve a read/trigger counterexample that defeats write-set diagnostics, and connect scheduler assumptions to replay claims.

Compensation and durable execution. The Compensatable footprint is an application of the saga lineage, where a long-lived transaction is decomposed and partial completion is amended by compensating transactions [7]. This paper adds no new compensation theory. It asks whether a fork's retained evidence shows that the required compensation or reconciliation actually occurred.

Temporal replays an ordered event history, requires deterministic workflow decisions, and reuses recorded activity results rather than repeating external work [19]. Azure Durable Task similarly uses event sourcing and constrains orchestrator code to deterministic APIs [12]. DBOS checkpoints

steps, requires deterministic workflow structure and idempotent steps, and does not re-execute a completed checkpoint [3]. Restate journals side-effecting operations and results so completed steps can be skipped on recovery [16]. These systems are strong industrial precedents for "deterministic replay after effect results are fixed." Our delta is not the mechanism; it is a per-cut, property-relative license that distinguishes projection, zero-call execution, continuation in a supplied environment, and a counterfactual-world observation.

LangGraph exposes checkpoint replay and fork operations. Its documentation states that nodes before a checkpoint are skipped while later nodes re-execute, including LLM and API calls [8]. This is precisely why a product-level "time travel" operation needs an effect-boundary contract: state branching and external re-execution are separate questions.

Agent calculi and mechanized semantics. λ_A gives a typed lambda calculus for agent composition with oracle calls, bounded fixpoints, probabilistic choice, and mutable environments, with machine-checked type-safety and bounded-termination results [9]. We study runtime event logs, schedule-sensitive settlement, and effect-boundary cuts rather than composition-time typing.

LLMbda models agent conversations, fork and clear operations, code generation, and labeled information flow. Its central result is machine-checked termination-insensitive probabilistic noninterference [6]. Our work does not claim noninterference or prompt-injection safety. Schlapbach formalizes Schema-Guided Dialogue and MCP in a process calculus and studies protocol-level bisimilarity and expressivity [17]; we instead examine effect evidence within a runtime trace.

Runtime substrates and governance. Agent libOS supplies a capability-controlled runtime with process identity, memory, skills, checkpoints, restore, fork, and commit [21]. It demonstrates operational mechanisms and a safety benchmark. Our question is which evidentiary conditions make a particular event-log cut sound for a named observation.

McCann's Effect-Transparent Governance formalizes handler-mediated effect governance in Rocq and proves semantic preservation, expressive minimality, and decidability boundaries [10]. We concede minimality and use an effect-transparent boundary as an executable assumption. AgentSpec defines trigger, predicate, and enforcement rules for agent safety [20]. Its ICSE 2026 contribution is runtime policy enforcement, not replay licensing.

ActiveGraph. ActiveGraph makes the event log authoritative and presents deterministic replay, cheap forks, and lineage as architectural properties [13]. This paper is explicitly a follow-on. It preserves the fixed-log projection result, distinguishes canonical ordering from schedule independence, and reframes forking as a property-indexed semantic contract. The public implementation and downstream artifacts supply the validation substrate, while their shared lineage limits external generalization.

Bounded novelty claim. After accounting for active rules, sagas, durable workflows, agent calculi, and checkpoint products, the remaining claim is intentionally small. We are not aware of prior work that jointly preserves runtime-order counterexamples, partitions effect evidence along footprint/source/lifecycle dimensions, assesses every historical cut under multiple fork properties, and checks those contracts against both archived traces and observed forks from a deployed event-sourced agent runtime. The empirical corpus is ActiveGraph-family and the search is not systematic; both limits are explicit.

9 Discussion

9.1 "The log replayed" is not one assertion

Figure 1 states the hierarchy early because it is the paper's organizing claim. None of its relations automatically implies all the others. The local executive study has equal decisions in two thirds of pairs and equal ordered paths in none. The Synthetic Players capsule passes an external byte-exact verifier but its individual logs contain no verification attestation, so they remain Observed under a log-alone grade. Every prefix projects, but cuts through requests fail execution continuation. The public bridge fixture then shows a different predicate again: one committed recorded oracle result is inherited across a fork with zero new calls under a verified fixture environment. These are not contradictions; they are different predicates.

9.2 Cheap fork is semantic before it is fast

Sharing an immutable prefix is computationally cheap. The harder question is what the child is allowed to do next. A runtime that advertises `fork(k)` should specify at least:

- the observation under which parent prefix and child are equivalent;
- the ordering authority for the prefix;
- whether a cut may split a request from its outcome;
- how recorded results are served without re-execution;
- which retained external effects are inherited;
- whether the child receives an isolated/snapshotted environment or the already-mutated source environment;
- the treatment of external effects in a retrospectively discarded suffix.

Without these clauses, "cheap" describes storage cost while leaving semantic cost undefined. With them, the implementation can refuse unsafe cuts, require an idempotency key, request compensation, select a snapshot, or downgrade the claim to ProjectionReplay.

A proposed copyable API contract is:

$$\mathsf{fork}(L, k, P) \to \mathsf{Ok}(\mathit{child}, \mathit{receipt}) \mid \mathsf{Refuse}(\mathit{reason}) \mid \mathsf{Conditional}(\mathit{obligation}). \tag{9}$$

The executable assessment represents a Conditional payload with the closed union

$$\begin{aligned}
\mathsf{Obligation} ::= {} & \mathsf{ServeRecordedResultWithoutReexecution}(effectId) && (10)\\
\mid {} & \mathsf{ResolveUnknownInheritedFootprint}(effectId) && (11)\\
\mid {} & \mathsf{ReconcileDiscardedEffect}(effectId) && (12)\\
\mid {} & \mathsf{ApplyCompensation}(effectId) && (13)\\
\mid {} & \mathsf{ReconcileFailedEffect}(effectId). && (14)
\end{aligned}$$

The proposed receipt binds the parent, cut, observation P, prefix hash, environment premise, inherited effect identities, and the typed obligation set. The F# assessor computes findings and this

obligation set. The companion activegraph-bridge v0.2 implementation now emits a hash-bound receipt and verifies an HMAC-authenticated target-environment claim under a caller-configured trust root; the public post-oracle fixture exercises that path. This is a conformance discharge mechanism, not a production identity or provider attestation. In particular, a cut through Requested is refused unless an explicit recovery policy changes the requested property.

9.3 Deterministic ordering can conceal non-confluence

FIFO and registration order are useful engineering contracts. They make one execution repeatable and improve debuggability. But they can turn an accidental ordering dependency into a stable behavior, making ordinary replay tests pass while schedule independence remains false. This matters when a runtime later parallelizes handlers, changes relation storage, distributes a queue, or moves from one database backend to another.

The appropriate response need not be to demand full confluence. Some systems may intentionally make order semantic. The contract should then name the canonical order, preserve it in the log, and reject backends that cannot supply it. Systems that want schedule independence need a stronger behavior non-interference discipline. The current work distinguishes the choices but does not prescribe one universally.

9.4 The log should carry its own licenses

The gap between capsule verification and log-alone grade suggests a design principle: evidence needed to authorize later replay or fork should be retained with the run or its signed sidecar. A minimum receipt should include canonical sequence, run identity, parent and cut lineage, effect request identity, footprint annotation, outcome linkage, response content hash, commitment or failure evidence, reconstruction method, and verification result.

This does not require every event to be trusted merely because it is present. The event type must be recognized, the producer or signature may need authentication, and cross-event integrity must be checked. The present F# artifact enforces structural integrity and exact recognized types; the companion bridge adds an HMAC-signed sidecar under a configured trust root. Production use still needs an operationally protected key, authoritative environment identity, rotation and revocation policy, and a freshness contract. A future asymmetric or transparency-backed evidence capsule could make that trust portable across implementations and organizations.

9.5 The graph may be a projection, but the scheduler remains real

Replayed domain state can be treated as a projection. That reduction still omits two operational facts. Once multiple behaviors react, event and activation order become part of the operational semantics unless a confluence condition removes them. Once effects leave the process, the external world becomes a second state that the event projection can describe but not reverse.

Thus reducing the runtime to a pure fold is useful but incomplete. The deeper object is an event-producing transition system plus a scheduler, an effect interpreter, and evidence linking interpreter outcomes back to the log. The graph may be derived; the ordering and effect contracts cannot be wished away.

9.6 Operational recommendations

The results suggest concrete contracts for event-sourced agent runtimes:

1. Treat production order as an explicit scheduler policy and test at least event-order and activation-order perturbations.

2. Record behavior read, write, emission, and trigger dependencies if schedule independence is a goal; a write set alone is insufficient.

3. Separate replay source from external footprint. "Cached" does not mean "never happened" or "safe to repeat."

4. Make request and terminal outcome a one-to-one integrity boundary. Refuse a cut through Requested or Unknown without an owner-selected recovery policy.

5. Give every fork API a property parameter or an equivalently explicit mode. Return a receipt, refusal, or conditional obligation rather than a single unqualified `safe` Boolean.

6. Preserve verification and reconstruction evidence inside a portable, hash-bound run capsule.

7. Distinguish actual observed forks from every-cut counterfactual tests. The former show where users forked; the latter reveal untested hazards.

These recommendations are compatible with F#, C#, Python, Rust, or TypeScript. The conformance vectors intentionally encode events and expected verdicts in language-neutral JSON so another implementation can challenge the semantics.

10 Limitations and Threats to Validity

No mechanized proof. The propositions and contracts are encoded as executable properties and checked over finite generators and artifacts. A passing FsCheck property is not a proof over all behaviors, schedulers, or logs. Future work may mechanize selected definitions after the operational contracts stabilize.

No minimal confluence condition. We falsify general confluence and declared-write disjointness. We check a restricted isolated-writer family. We do not establish a necessary or sufficient general condition over reads, writes, emissions, triggers, and state domains. Calling the current restricted condition "the" restoring condition would overstate the result.

Limited scheduler space. The executable suite uses four deterministic scheduler extremes. They expose the preserved counterexamples but do not enumerate every interleaving for an arbitrary behavior set. Relation ordering is discussed from source audit but not modeled as a separate production activation dimension. Seeded randomized and bounded exhaustive interleaving exploration remain future checks.

Adapter assumptions. Real-log classification relies on explicit event names and payload fields. The adapter is deliberately conservative, but mistakes remain possible. Model calls are treated as OneShot for cost/oracle semantics; another property might classify a read-only deterministic local model differently. A derived content hash proves captured bytes, not provider authenticity, commitment, or idempotence.

The generic F# effect descriptor and legacy corpus adapters have no per-effect observable-set label. Their CounterfactualWorld results therefore use the selected profile's entire recognized

external surface as $\Omega_{profile}$. The public bridge v0.2 receipt records observable tags, but we do not retroactively impute them to older corpus events. Narrower corpus claims require adapter-supplied tags and new conformance vectors.

Unknown footprint fails closed only after the adapter recognizes an event as an effect. Unclassified source events remain reported signals. The all-cut Sound counts therefore depend on an audited profile declaration that those event types do not hide external interactions. A newly introduced effect type can make an old verdict false until the adapter is updated.

Partial failure and compensation scope. Failed remote operations may have partially committed before reporting an error. The assessor conservatively conditions or blocks discarded failures by footprint, but it cannot inspect the remote system. Likewise, successful compensation does not erase a temporal exposure window or third-party secondary effects. A caller whose Ω includes those histories needs stronger reconciliation evidence than the current effect descriptor records.

Whole-log malformedness policy. The current assessor treats structural malformedness anywhere in the source log as a blocker for every cut, including projection. This is fail-closed and appropriate for the evidence study, but it is stricter than a theorem about an independently authenticated well-formed prefix. The projection suffix identity is therefore stated only for well-formed ordered logs.

Synthetic content. The Synthetic Players capsule contains archived experimental runs, but the present analysis concerns runtime traces rather than human ground truth. The public bridge fixture and local executive study use deterministic offline fixtures or mocks and explicitly make no real-model performance claim. The executive-study agreement numbers illustrate equivalence relations, not predictive validity.

Fork coverage. The 121 observed forks are useful positive evidence, but all cut at `round.played` and retain no classified external request. No observed fork in the public capsule crosses a recorded oracle or irreversible effect boundary, and all 160,076 Sound corpus ExternalContinuation cuts precede the first classified effect. The companion public conformance fixture now covers one actual bridge fork after a committed recorded oracle effect. Its receipt verifies zero inherited external calls and an HMAC-authenticated fixture environment under a configured trust root. This closes the prior absence of a public no-reexecution mechanism test, but only for one deterministic offline oracle, one child, and a deliberately published fixture key. It does not establish real-provider behavior, production attestor identity, environmental fidelity, or prevalence across production forks.

Executive-study provenance. The separate 24-run executive-study artifact is locally content-addressed and bound to local source and release-bundle commits with a standalone verifier. Its migration-lab repository still has no public remote, so a fresh external reviewer cannot fetch those revisions. The result remains illustrative and excluded from the abstract until the bundle is published. This limitation is particularly important because provenance is part of the paper's thesis; it does not apply to the public post-oracle bridge fixture above.

Production behavior inventory. The ActiveGraph source establishes FIFO and registration-order scheduling, but we did not recover a complete deployed behavior dependency inventory. We therefore cannot say whether production behaviors satisfy a future non-interference condition. Static source inspection of the scheduler is not production confluence validation.

No external runtime corpus. The public corpus, observed forks, bridge implementation, and local executive study all descend from ActiveGraph. The language-neutral vectors support other implementations, but no LangGraph, Temporal, or other independent runtime trace has yet been normalized. Generalization beyond the source family is therefore conceptual rather than empirically demonstrated.

No authorization or information-flow result. Effect descriptors classify replay and environmental conditions; they do not grant capabilities or prove that a caller is authorized. The artifact also does not track secrecy labels or prove noninterference. Those topics are covered by distinct systems and calculi and remain explicit non-goals.

No performance benchmark. The all-cut implementation is efficient enough for the reported corpora, but we make no throughput, storage, or latency claim. "Cheap" in this paper is analyzed semantically. A separate benchmark would be needed for production performance.

11 Conclusion

Deterministic projection of a fixed log is valuable, but it is the beginning of an agent-runtime replay contract rather than the end. Reactive settlement can diverge, a stable production order can mask non-confluence, and disjoint declared writes do not exclude schedule-sensitive reads or trigger interference. A fork can preserve its trace prefix while failing to reproduce execution or the external world.

The executable contract makes those distinctions checkable. Effect footprint, replay source, and lifecycle answer separate questions. Fork safety is indexed by an observation: projection, strict execution replay, continuation in a prefix-reconstructed environment, or counterfactual-world equivalence. Replay grades summarize retained evidence, while cut-level assessment decides whether a particular branch meets the property-specific preconditions.

The runtime artifacts support both positive and negative conclusions. Across 5,540 public capsule logs, every one of 317,296 projected prefixes is sound under the stated projection observation. Across 121 observed forks, 16,625 shared-prefix events show zero structural mismatches modulo expected lineage fields, and the observed corpus cuts avoid external requests. Yet 36,251 hypothetical cuts split model requests and are unsound for continuation, while thousands of discarded oracle interactions prevent an unchanged-world claim. A separate public offline bridge fixture crosses one committed recorded oracle boundary and verifies zero inherited external calls under an authenticated conformance environment. A local executive study further illustrates that decision and path equivalence can separate, but it remains outside the public headline evidence until its commit-bound bundle is published.

The practical conclusion is concise: replay is a family of assertions, and a fork is sound only relative to one of them. Event-sourced agent runtimes should record the evidence that licenses each assertion, expose ordering assumptions, and refuse to turn a cheap prefix copy into an unqualified promise about execution or the world.

References

[1] Alexander Aiken, Joseph M. Hellerstein, and Jennifer Widom. Static analysis techniques for predicting the behavior of active database rules. *ACM Transactions on Database Systems*, 20(1):3–41, 1995.

[2] Jérémie Chassaing. Functional event sourcing decider. https://thinkbeforecoding.com/post/2021/12/17/functional-event-sourcing-decider, 2021. Accessed 2026-08-28.

[3] DBOS, Inc. Dbos architecture. https://docs.dbos.dev/architecture, 2026. Accessed 2026-08-28.

[4] Elm Project. The elm architecture. *An Introduction to Elm*, https://guide.elm-lang.org/architecture/. Accessed 2026-08-28.

[5] Martin Fowler. Event sourcing. https://www.martinfowler.com/eaaDev/EventSourcing.html, 2005. Accessed 2026-08-28.

[6] Zac Garby, Andrew D. Gordon, and David Sands. The llmbda calculus: Ai agents, conversations, and information flow. *arXiv preprint arXiv:2602.20064*, 2026.

[7] Hector Garcia-Molina and Kenneth Salem. Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, pages 249–259, 1987.

[8] LangChain. Use time travel: Replay and fork. https://docs.langchain.com/oss/python/langgraph/use-time-travel, 2026. Accessed 2026-08-28.

[9] Qin Liu. λ_a: A typed lambda calculus for llm agent composition. *arXiv preprint arXiv:2604.11767*, 2026.

[10] Alan L. McCann. Effect-transparent governance for ai workflow architectures: Semantic preservation, expressive minimality, and decidability boundaries. *arXiv preprint arXiv:2605.01030*, 2026.

[11] George H. Mealy. A method for synthesizing sequential circuits. *Bell System Technical Journal*, 34(5):1045–1079, 1955.

[12] Microsoft. Durable task code constraints. https://learn.microsoft.com/en-us/azure/durable-task/common/durable-task-code-constraints, 2026. Accessed 2026-08-28.

[13] Yohei Nakajima. The log is the agent: Event-sourced reactive graphs for auditable, forkable agentic systems. *arXiv preprint arXiv:2605.21997*, 2026.

[14] M. H. A. Newman. On theories with a combinatorial definition of equivalence. *Annals of Mathematics*, 43(2):223–243, 1942.

[15] Norman W. Paton and Oscar Díaz. Active database systems. *ACM Computing Surveys*, 31(1):63–103, 1999.

[16] Restate. Key concepts: Durable execution. https://docs.restate.dev/foundations/key-concepts, 2026. Accessed 2026-08-28.

[17] Andreas Schlapbach. Formal semantics for agentic tool protocols: A process calculus approach. *arXiv preprint arXiv:2603.24747*, 2026.

[18] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-free replicated data types. In *Stabilization, Safety, and Security of Distributed Systems*, volume 6976 of *Lecture Notes in Computer Science*, pages 386–400. Springer, 2011.

[19] Temporal Technologies. Temporal workflow: How workflow replay works. `https://docs.temporal.io/workflows`, 2026. Accessed 2026-08-28.

[20] Haoyu Wang, Chris Poskitt, and Jun Sun. Agentspec: Customizable runtime enforcement for safe and reliable llm agents. In *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026.

[21] Yingqi Zhang. Agent libos: A runtime substrate for capability-controlled self-evolving llm agents. *arXiv preprint arXiv:2606.03895*, 2026.